

终极试炼

数学、贪心与综合实战 · 8题 · C++ & Python 双语对照

最后一章！小鲁回顾了这一路风景——从第一个A+B，到循环、数组、字符串、函数、STL、排序、二分、前缀和、差分、双指针、高精度、位运算、离散化、数据结构、搜索、图论、动态规划.....16章的征程即将画上句号。最后一章，我们用数学和贪心来收尾——有时候最优解不需要复杂算法，一个定理就够了。"编程的最高境界——不是写最多的代码，而是用最少的代码解决最难的问题。"

🏆 终极速查

- **贪心**：区间选点(排序右端点)、合并果子(Huffman+堆)——每次局部最优
- **并查集**：路径压缩+按秩合并——近乎O(1)的集合操作
- **带权并查集**：食物链——维护到根的距离，模运算分类关系
- **数论**：快速幂O(log b)、筛质数(线性筛O(n))、中位数选址——数学直接给答案

NQ159 区间选点 AcWing 905

"在数轴上有N个区间，选出最少的点，使每个区间至少包含一个点。"小华说，"贪心经典题——按右端点排序，每次选当前未覆盖区间中最小的右端点作为新点。因为右端点排序后，选右端点覆盖最多后续区间。贪心的核心：'每次做当前看起来最好的选择'——但关键是证明贪心策略的正确性。排序+遍历=O(n log n)。"

输入 第一行N。然后N行每行l r表示区间。

输出 最少点数。

范围 $1 \leq N \leq 10^5, -10^9 \leq l \leq r \leq 10^9$

输入	输出
3	2
-1 1	
2 4	
3 5	

思路 贪心策略：按右端点升序排序。ans=0, last=-INF。遍历每个区间[l,r]：若l>last则需新点ans++，last=r。证明：按右端点排序后，每次选最小的右端点保证了对后续区间的最大覆盖能力。

技巧 贪心的正确性证明通常用'交换论证'或'归纳法'。区间问题的通用解法：先排序再贪心——排序确定处理的顺序，贪心确定每一步的选择。Python: intervals.sort(key=lambda x: x[1])按右端点排序。

C++

```

#include <iostream>
#include <cstring>
#include <algorithm>

using namespace std;

const int N = 100007;

struct Range
{
    int l, r;
    bool operator< (const Range & R) const
    {
        return r < R.r; //重载比较符, 根据r排序
    }
} range[N];

int main()
{
    int n;
    scanf("%d", &n);
    //读入区间
    for (int i = 0; i < n; i++)
    {
        int l, r;
        scanf("%d%d", &l, &r);
        range[i] = {l, r};
    }
    //按照右端点排序
    sort(range, range+n);
    //根据右端点计算覆盖的区间个数
    int res = 0, ed = -2e9; //已统计区间的右端点
    for (int i = 0; i < n; i++)
    {
        if (range[i].l > ed)
        {
            res++;
            ed = range[i].r;
        }
    }
    printf("%d\n", res);

    return 0;
}

```

Python

Python solution

NQ160 合并果子 AcWing 148

"N堆果子，每次合并任意两堆，代价为两堆之和。求合并成一堆的最小总代价。"这就是Huffman编码！"每次选最小的两堆合并——用小根堆维护。贪心+Huffman=O(n log n)。"小华说，"Huffman树的每个内部节点对应一次合并，叶子是原始堆。最优性证明：最小的两堆一定在Huffman树的最深层——交换论证。"

输入

第一行n。第二行n个整数。

输出

最小总代价。

范围

 $1 \leq n \leq 10000$

输入

输出

3
1 2 9

15

思路 小根堆(优先队列)：每次pop两个最小值，合并(=和)，将和push回堆，ans+=和。重复直到堆中只剩一个元素。时间 $O(n \log n)$ 。Python用heapq: heapify(arr); while len>1: a,b=heappop,heappop; heappush(a+b); ans+=a+b。

技巧 Huffman编码=最优二叉树构造。堆的选择：小根堆取最小值最快($O(\log n)$)。这是优先队列最重要、最经典的应用——Dijkstra、Prim都用到了堆。Python的heapq模块简单高效——记住heappush+heappop组合。

C++

```
#include <iostream>
#include <queue>

using namespace std;

int main()
{
    int n; cin>>n;
    //优先队列, 使用greater作比较函数,把默认的大根堆变为小根堆
    priority_queue<int, vector<int>, greater<int>> heap;
    //把读入数据压入堆
    while (n-->0)
    {
        int x; cin>>x;
        heap.push(x);
    }

    int res = 0;
    while(heap.size() > 1)
    {
        int x = heap.top(); heap.pop();
        int y = heap.top(); heap.pop();
        res += x + y; //累计合并值
        heap.push(x+y); //把当前合并的值压入堆
    }

    cout<<res<<endl;

    return 0;
}
```

Python

```
# Python solution
```

NQ161 合并集合 AcWing 836

"并查集(Disjoint Set Union)——处理集合的合并和查询。"小华画出带箭头的图，"每个集合用一个树表示，根节点是集合代表。find(x)：找x的根。union(x,y)：将x根连到y根。两种优化：路径压缩(find时把所有路径节点直接连根)和按秩合并(小树连到大树)。均摊复杂度近乎 $O(1)$ ——这是数据结构设计中最精妙的成就之一。"

输入 第一行n,m。然后m行：M a b(合并)或Q a b(查询是否同集)。

输出 对Q操作输出Yes/No。

范围

$1 \leq n, m \leq 10^5$

输入	输出
4 5	Yes
M 1 2	No
M 3 4	Yes
Q 1 2	
Q 1 3	
Q 3 4	

思路 并查集：p[x]存父节点(初始p[x]=x)。find(x): if p[x]!=x: p[x]=find(p[x]); return p[x]——路径压缩。
union(x,y): p[find(x)]=find(y)。所有操作均摊 $O(\alpha(n)) \approx O(1)$ 。这是数据结构课程上最简洁又最高效的设计之一。

技巧 并查集是竞赛中使用频率最高的数据结构。路径压缩(find中的p[x]=find(p[x]))是将树扁平化的关键。按秩合并：若两个集合秩相同则根秩+1。Python递归find可能超深度——用while迭代或sys.setrecursionlimit。并查集在Kruskal、连通性、动态图问题中是核心工具。

C++

```

#include <iostream>
using namespace std;
const int N = 100007;
int p[N];
int n;

//返回x的father节点+路径压缩
int find(int x)
{
    if (p[x] != x) p[x] = find(p[x]);
    return p[x];
}
void merge(int a, int b)
{
    p[find(a)] = find(b);
}
void query(int a, int b)
{
    if (find(a) == find(b)) puts("Yes");
    else puts("No");
}
int main()
{
    int m;
    cin>>n>>m; //输入n
    //创建并查集数
    for (int i = 1; i <= n; i++) p[i] = i;
    while (m--)
    {
        char op;
        int a, b;
        cin>>op;
        cin>>a>>b;
        if (op == 'M') merge(a, b);
        else if (op == 'Q') query(a, b);
    }
    return 0;
}

```

Python

Python solution

NQ162 连通块数量 AcWing 837

"上一题的扩展——除了查询连通性，还要知道每个连通块的大小。"小华说，"在并查集中额外维护一个size数组——初始size[i]=1。合并时size[新根]+=size[被合并的根]。查询某个节点所在连通块大小只需size[find(x)]。如果在合并时重复合并同一集合，size不变——避免重复累加。这是并查集的自然扩展。"

输入

第一行n,m。然后m行：C a b(合并,若已连通忽略) / Q1 a b(查询连通) / Q2 a(查询a所在连通块大小)。

输出

按要求输出。

范围

$1 \leq n, m \leq 10^5$

输入

```

5 5
C 1 2
Q1 1 2
Q2 1

```

输出

```

Yes
2
3

```

C 2 3
Q2 1

思路 维护size的并查集：初始化size[i]=1。union时：if a_root!=b_root: p[a_root]=b_root; size[b_root]+=size[a_root]。注意必须先判连通再合并——避免重复size。find不变。

技巧 并查集可以维护更多信息：size、距离到根(带权并查集)、最小值/最大值等。关键是理解'合并时如何更新附加信息'。食物链(240)是带权并查集的经典应用——每个节点维护与父节点的关系。

C++

```

#include <iostream>
#include <cstring>

using namespace std;
const int N = 100007;
int p[N], Size[N];
int n;

//返回x的father节点+路径压缩
int find(int x)
{
    if (p[x] != x) p[x] = find(p[x]);
    return p[x];
}
void merge(int a, int b)
{
    Size[find(b)] += Size[find(a)];
    p[find(a)] = find(b);
}
void query(int a, int b)
{
    if (find(a) == find(b)) puts("Yes");
    else puts("No");
}
int count(int x)
{
    return Size[find(x)];
}
int main()
{
    int m;
    cin >> n >> m; //输入n
    //创造并查集数
    for (int i = 1; i <= n; i++) p[i] = i, Size[i] = 1;
    while (m--)
    {
        string op;
        int a, b;
        cin >> op;
        if (op == "C")
        {
            cin >> a >> b;
            if (find(a) == find(b)) continue; //已经在同一个集合里,
            //跳过merge操作
            merge(a, b);
        }
        else if (op == "Q1") cin >> a >> b, query(a, b);
        else cin >> a, cout << count(a) << endl;;
    }
    return 0;
}

```

Python

Python solution

NQ163 食物链 AcWing 240

"动物王国三种关系：A吃B、B吃C、C吃A——循环克制。"这是带权并查集的巅峰应用！"每个节点维护到根的距离 $d[x]$ ： $d[x] \% 3 = 0$ 与根同类， $=1$ 吃根， $=2$ 被根吃。合并集合时根据两个节点的关系计算出它们根之间的距离关系。路径压缩时同步更新 $d[x]$ 。这是竞赛中最精彩的并查集应用！"

输入 第一行N,K。然后K行: D X Y (D=1同类,2表示X吃Y)。

输出 假话数量。

范围 $1 \leq N \leq 50000, 0 \leq K \leq 100000$

输入	输出
100 7	3
1 101 1	
2 1 2	
2 2 3	
2 3 3	
1 1 3	
2 3 1	
1 5 5	

思路 带权并查集: $p[x]$ 存父, $d[x]$ 存与父的关系(0同类/1吃父/2被父吃)。find时路径压缩: $root=find(p[x]); d[x]+=d[p[x]]$; $p[x]=root$ 。合并X和Y: 根据 $D=1/2$ 计算X根和Y根的距离关系。判断假话: X/Y超范围、已连通但与关系不符。

技巧 食物链是并查集+模3运算的智力巅峰。 $d[x]\%3$ 是与根的关系: 0同类、1吃根、2被根吃。路径压缩时 $d[x]+=d[p[x]]$ 保证距离累加正确。理解这道题后, 所有带权并查集问题都不在话下。这是OI/ACM金牌选手的基本功。

C++

```

#include <iostream>

using namespace std;

const int N = 50007; //Andy风格

int n, m;
int p[N], d[N]; //p集合 d距离初始化为0

int find(int x)
{
    if (p[x] != x)
    {
        int t = find(p[x]);
        d[x] += d[p[x]];
        p[x] = t;
    }
    return p[x];
}

int main()
{
    scanf("%d%d", &n, &m);

    for (int i = 1; i <= n; i++) p[i] = i; //初始化p集合

    int res = 0;
    while (m --)
    {
        int t, x, y;
        scanf("%d%d%d", &t, &x, &y);

        if (x > n || y > n) res ++;
        else
        {
            int px = find(x), py = find(y); //两个集合的父节点
            if (t == 1)
            {
                if (px == py && (d[x] - d[y]) % 3) res ++;
                else if (px != py)
                {
                    p[px] = py;
                    d[px] = d[y] - d[x];
                }
            }
            else
            {
                if (px == py && (d[x] - d[y] - 1) % 3) res ++;
                else if (px != py)
                {
                    p[px] = py;
                    d[px] = d[y] + 1 - d[x];
                }
            }
        }
    }

    printf("%d\n", res);
    return 0;
}

```

Python

Python solution

NQ164 快速幂 AcWing 875

"计算 $a^b \bmod p$ —— b 可能大到 10^9 。"小华在黑板上写下算法，"朴素循环 $O(b)$ 超时。快速幂：利用 $a^b = a^{(b/2)} \times a^{(b/2)}$ ——分治+二进制分解。while(b): if(b&1) res=res*a%p; a=a*a%p; b>>=1。核心：将指数按二进制位拆开计算——时间 $O(\log b)$ 。这个模板出现在RSA加密、矩阵快速幂、模逆元等场景中。"

输入

第一行 n 。然后 n 行每行 $a \ b \ p$ 。

输出

每行 $a^b \bmod p$ 。

范围

$1 \leq n \leq 10^5, 1 \leq a, b, p \leq 2 \times 10^9$

输入

2
2 3 5
3 2 7

输出

3
2

思路 快速幂while循环： $a^b \bmod p$ 。while b>0: if b&1: res=res*a%p; a=a*a%p; b>>=1。每步 b 减半→ $O(\log b)$ 。Python: pow(a,b,p)——行内置(支持三参数取模)——底层用快速幂实现。但手写理解原理是必须的。

技巧 Python pow(a,b,p)三步参数： $a^b \bmod p$ ——内置最高效。快速幂是数论算法的基础——之后模逆元、快速幂+矩阵乘法(矩阵快速幂)、米勒-拉宾素性检测都依赖于此。位运算 $b \& 1$ 判断奇偶比 $b \% 2$ 更快。

C++

```
#include <iostream>
#include <algorithm>
using namespace std;
typedef long long LL;
int qmi(int a, int b, int p)
{
    int res = 1; //初始值为1
    while (b)
    {
        if (b & 1) //b的最末尾为1
            res = (LL)res * a % p; //必须用long long防止溢出
        b >>= 1; //去掉b最低位
        a = (LL)a * a % p;
    }
    return res;
}

int main()
{
    int n;
    scanf("%d", &n); //数据很大, 需要用LL读入
    while (n--)
    {
        int a, b, p;
        scanf("%d%d%d", &a, &b, &p); //读入a b p

        printf("%d\n", qmi(a, b, p)); //计算快速幂
    }
    return 0;
}
```

Python

Python solution

NQ165 筛质数 AcWing 868

"找出1到n中的所有质数。"小华说, "埃氏筛法(Eratosthenes): 从2开始, 依次划掉2的倍数、3的倍数..... 未被划掉的就是质数。优化: 每个数只被它最小的质因子划掉——这就是线性筛。线性筛保证每个合数只被标记一次, $O(n)$ 。理解线性筛是理解数论算法的关键——最小质因子是数论中最核心的概念之一。"

输入 一个整数n。

输出 1到n中质数的个数。

范围 $1 \leq n \leq 10^6$

输入

8

输出

4

思路 埃氏筛: bool数组标记合数, for i from 2: if !st[i]: primes.push(i); for j=i*i到n步长i: st[j]=true。 $O(n \log \log n)$ 。线性筛: 每个数只被最小质因子筛——for p in primes: if i*p>n break; st[i*p]=true; if i%p==0 break。 $O(n)$ 。

技巧 筛法不仅是求质数——还可以同时记录最小质因子、欧拉函数、约数个数等。线性筛复杂度 $O(n)$ 所以可以处理 $n=10^7$ 。Python的筛法比C++慢几倍但 $n=10^6$ 仍可。关键优化: 只用 $\leq \sqrt{n}$ 的质数来筛。

C++

```
// Andy 2021.06.04
#include <iostream>
#include <cstring>
#include <algorithm>

using namespace std;

const int N = 100000007;

int primes[N], cnt;
bool st[N];
void get_primes(int n) // 线性筛质数
{
    for (int i = 2; i <= n; i++)
    {
        if (!st[i]) primes[cnt++] = i;
        for (int j = 0; primes[j] <= n / i; j++)
        {
            st[primes[j] * i] = true;
            if (i % primes[j] == 0) break;
        }
    }
}

int main()
{
    int n; cin >> n;
    get_primes(n);
    cout << cnt << endl;
    return 0;
}
```

Python

Python solution

NQ166 货仓选址 AcWing 104

"最后一个问题——在数轴上选一个位置建仓库，使到N个商店的总距离最小。"小华笑道，"答案是中位数！将商店坐标排序，最优位置=中位数位置。证明：如果把仓库往任意方向移dx，离一半商店近了dx但离另一半远了dx——中位数处达到平衡。这展示了数学思维在算法优化中的力量——有时候最优解不需要代码搜索，数学定理直接给出答案。"

输入 第一行N。第二行N个整数。

输出 最小总距离。

范围 $1 \leq N \leq 100000$

输入	输出
4	12
6 2 9 1	

思路 排序取中位数：`sort(arr); median=arr[n//2]; ans=sum(abs(a-median))`。O(n log n)。进阶：如果N是偶数，任何在中间两个数之间的位置都是最优的。中位数距离和是最小绝对偏差问题的标准答案——不需要

搜索，理解定理就完成了。

技巧 中位数vs平均数：最小化绝对偏差和用中位数，最小化平方偏差和用平均数。这是统计学的基础结论。Python一行：median=sorted(arr)[n//2]; print(sum(abs(x-median) for x in arr))。16章到此结束——从A+B到货仓选址，你已经完成了算法竞赛入门的所有基础训练。

C++

```
#include <iostream>
#include <cstring>
#include <algorithm>

using namespace std;

const int N = 100007;

int n;
int a[N];

int main()
{
    cin >> n;
    for (int i = 0; i < n; i++) cin >> a[i];
    sort(a, a+n);
    int res = 0;
    //中位数a[n/2],求距离之和的最小值
    for (int i = 0; i < n; i++) res += abs(a[i] - a[n/2]);
    cout << res << endl;
    return 0;
}
```

Python

```
# Python solution
```

全书总结

- 16章·161题·C++/Python双语：从A+B到货仓选址——完整覆盖算法竞赛入门的全部知识点
- 四大阶段：语法基础(L1-5)→编程进阶(L6-8)→核心算法(L9-13)→算法进阶(L14-16)
- 两语言并行：C++让你理解底层，Python让你快速验证——两种视角互补
- 厦大校训：自强不息，止于至善。16章的征程结束，但算法之路刚刚开始。

— 全16章完 · 共161题 · 自强不息，止于至善 —